

PCB Pick-and-Place: Answers to Your Questions

Oct 4, 2026 · @Håvard Karlsen Solheim

The system picks through-hole components (capacitors, resistors, transformers) and sorts them into bins, as a concept for PCB recycling with off-the-shelf hardware. Short answers first; each is detailed in the numbered section below.

Short answers

1. **Data:** 152 annotated photos of our own, plus 1,296 images from a public PCB dataset (CC0). Our photos are split 107 train / 30 validation / 15 test; the public images are used for training only.
2. **Model:** YOLO26s (Ultralytics), the successor to YOLO11. An earlier prototype used YOLOv11s; the new model scores higher on the same test photos.
3. **Architecture:** RealSense D435i camera → YOLO26s detection and 3D localisation → MoveIt 2 motion planning → xArm6 with gripper, sorting each part into a bin by class. Block diagram in section 3.
4. **Plots and results:** detection accuracy (test mAP@50 = 0.84, 85% of parts found), the training curve, and the pick-and-place trajectory are in section 4. On the robot, 20 of 22 capacitor picks succeeded on 4 October 2026; resistors and transformers were not tested.

1. Data: which data, how many pictures, how it is split

The detector is trained on 1,403 images: 107 of our own photos plus 1,296 from the public PCB Component Detection dataset. Validation and test use only our own photos (30 and 15), so the reported scores describe our setup.

Split	Our photos	Public images	Our boxes: capacitor / resistor / transformer
Train	107 (each used 3×)	1,296	2,020 / 51 / 153
Validation	30	0	762 / 7 / 52
Test	15	0	253 / 8 / 29
Total	152	1,296	3,035 / 66 / 234

Our images. 152 annotated photos of the components, labelled in Roboflow (project [carm-yitb6/carm-capacitor-test-set](#), export v9, CC BY 4.0). 69 are 1280 × 720 px; the other 83 are larger photos, 2,000–4,000 px on the long side. The split is by photo, stratified so that resistors appear in both validation and test. Each training photo is repeated three times so that our images keep their share of the training mix against the public set.

Public images. [PCB Component Detection](#) (CC0, originally on [Kaggle](#)): top-down photos of populated PCBs. Its classes are mapped onto ours: Cap1–Cap4 → capacitor (4,832 boxes), Resistor → resistor (1,154), Transformer → transformer (2,547). MOSFET and varistor (MOV) boxes are removed, so the model learns to ignore those parts. Images with no labels (114) are skipped.

Preprocessing. Auto-orient, then resize to fit within 1,280 × 1,280 px with the aspect ratio kept. No augmented copies are stored. Most of our labels were drawn as polygons (91%); they are converted to axis-aligned bounding boxes.

Augmentation is applied on the fly during training (Ultralytics defaults): mosaic (off for the last 10 epochs), horizontal flip ($p = 0.5$), scale $\pm 50\%$, translation $\pm 10\%$, and hue/saturation/value jitter (0.015 / 0.7 / 0.4).

Resistors remain the rarest class in our own data: 66 boxes, against 3,035 capacitors.

2. Model: is it YOLO11?

Not YOLO11 itself: the deployed detector is YOLO26s (Ultralytics), the successor to YOLO11, fine-tuned from COCO-pretrained weights. An earlier prototype used a YOLOv11s model hosted on Roboflow; the comparison is at the end of this section.

Setting	Value
Architecture	YOLO26s object detection (small variant)
Initialisation	COCO-pretrained weights

Setting	Value
Input size	1,280 px (long side)
Epochs	150 maximum; early stopping after 30 epochs without improvement. Stopped at epoch 104; best weights from epoch 74
Batch size	8
Optimiser	AdamW, learning rate 0.00143 (chosen automatically by Ultralytics), mixed precision
Hardware and time	NVIDIA RTX 5070 (12 GB), 55 min
Software	Ultralytics 8.4.172, PyTorch 2.11 (CUDA 12.8), Python 3.10

Inference pipeline. The model runs locally on the robot PC's GPU inside the detection node. Each RealSense colour frame is passed whole at 1,280 px; one frame takes about 8 ms. Detections below 0.50 confidence are dropped; this threshold gives the best F1 on the test split. YOLO26 needs no separate non-maximum suppression step. Targets less than 8 mm apart in 3D are still merged, in case one part is reported twice.

The Roboflow prototype instead tiled each frame into 400×400 px slices (SAHI) to help with small parts. The full-resolution model runs on the whole frame instead.

Model comparison

Training at full resolution with the true aspect ratio gives the largest gain. Both YOLO26s rows are scored on the same 15 test photos.

Model	Training images	Test mAP@50	Test mAP@50-95	Capacitor recall
YOLO26s, full resolution (deployed)	Fit within 1,280 px, aspect kept	0.84	0.60	0.88
YOLO26s, 640 px	Stretched to 640×640	0.77	0.44	0.62
YOLOv11s on Roboflow (prototype)	Stretched to 640×640 , own data only	0.74	0.44	0.75

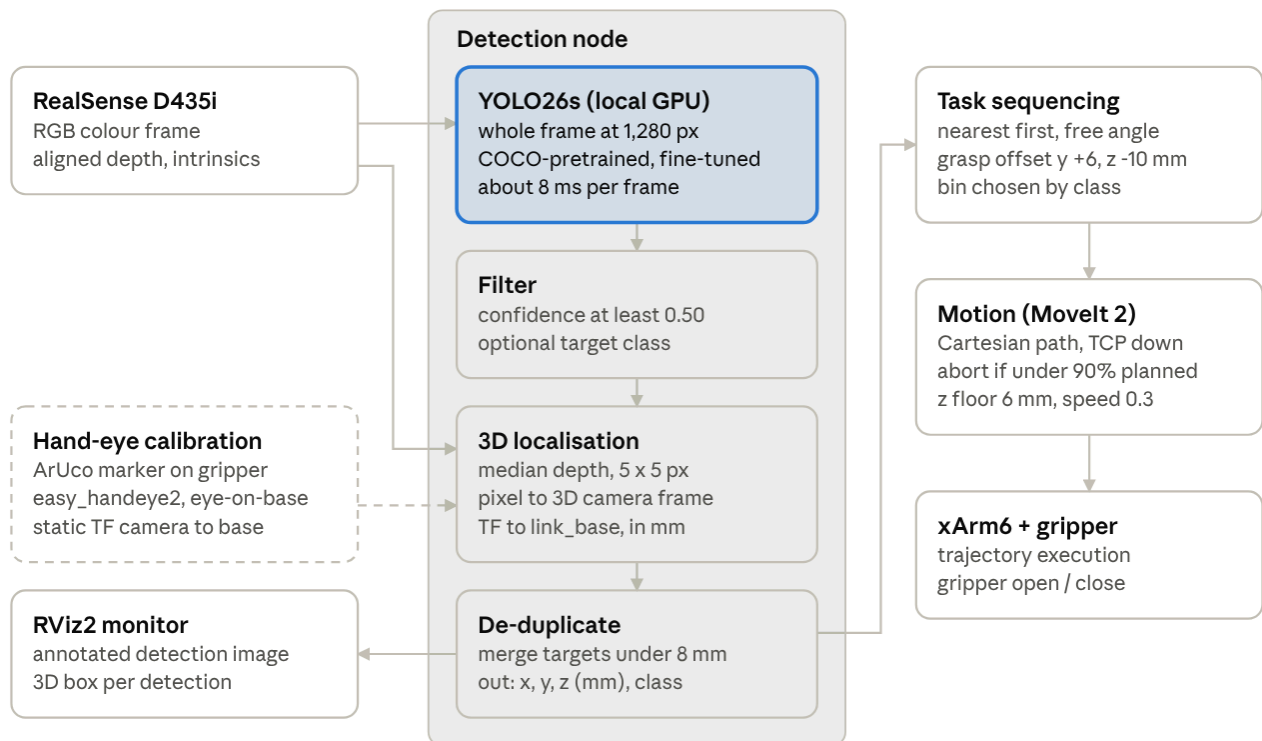
The prototype row was measured by Roboflow on its own 2-image test split (40 objects), so it is not directly comparable to the other two.

3. System architecture (block diagram)

Three ROS 2 parts run in one process: a detection node, a task sequencer and a MoveIt 2 motion client. The only offline step is the hand-eye calibration.

Each snapshot becomes robot-frame targets, picked with straight-line moves

Data flow for one detection cycle (key s pressed), ROS 2 Humble



Blue: the YOLO26s detector. Dashed: offline step, run once after mounting the camera.

System architecture · from main/main.py, detection_model/yolo_model.py, helpers/movement.py

Each pick runs six straight-line moves: above the part, down, grip, up, above the bin, down, release, up, then home at (130, 0, 300) mm. Collision objects for the camera pole and mount are added to the MoveIt planning scene at start-up.

Since 4 October the gripper also turns about the vertical axis before it comes down. Twelve angles are checked against the depth image, and the one where the open fingers land on free space is used; a part with no free angle is skipped until its neighbours are gone. Moves run at 0.3 of the planned speed, and the lowest grasp height is 6 mm above the table.

Hardware and software

Part	What is used
Robot	UFactory xArm6 with the xArm parallel gripper

Part	What is used
Camera	Intel RealSense D435i, fixed mount beside the arm (eye-on-base)
Middleware	ROS 2 Humble, MoveIt 2 (Cartesian path planning)
Detection	YOLO26s (Ultralytics), run locally on the GPU on the full frame at 1,280 px
Calibration	ArUco marker (DICT_6X6_1000, id 398, 115 mm) on the gripper, solved with easy_handeye2
Language	Python 3 (rclpy, OpenCV, NumPy, PyTorch, Ultralytics)

4. Plots: accuracy, trajectory and failure rates

Detection accuracy

On the 15 held-out test photos (290 objects) the model reaches $mAP@50 = 0.84$ and $mAP@50-95 = 0.60$, finding 85% of components. On the 30 validation photos (821 objects) it scores $mAP@50 = 0.85$ and $mAP@50-95 = 0.63$.

Class	Test objects	Precision	Recall	$mAP@50$	$mAP@50-95$
All	290	0.78	0.85	0.84	0.60
Capacitor	253	0.85	0.88	0.91	0.70
Resistor	8	0.76	0.88	0.81	0.49
Transformer	29	0.74	0.78	0.80	0.60

The resistor scores rest on 8 test objects and should be read with that in mind.

On the robot's own camera the result is narrower than these scores suggest. The camera looks at the table from the side, and on two test boards on 4 October it found the capacitors reliably but missed the transformers and most of the resistors. The robot runs below therefore use capacitors only.

Training curve

Validation mAP@50-95 peaks at 0.63 at epoch 74, then plateaus

YOLO26s at 1,280 px, 30 validation photos (821 objects). Dots per epoch; lines are 10-epoch means.



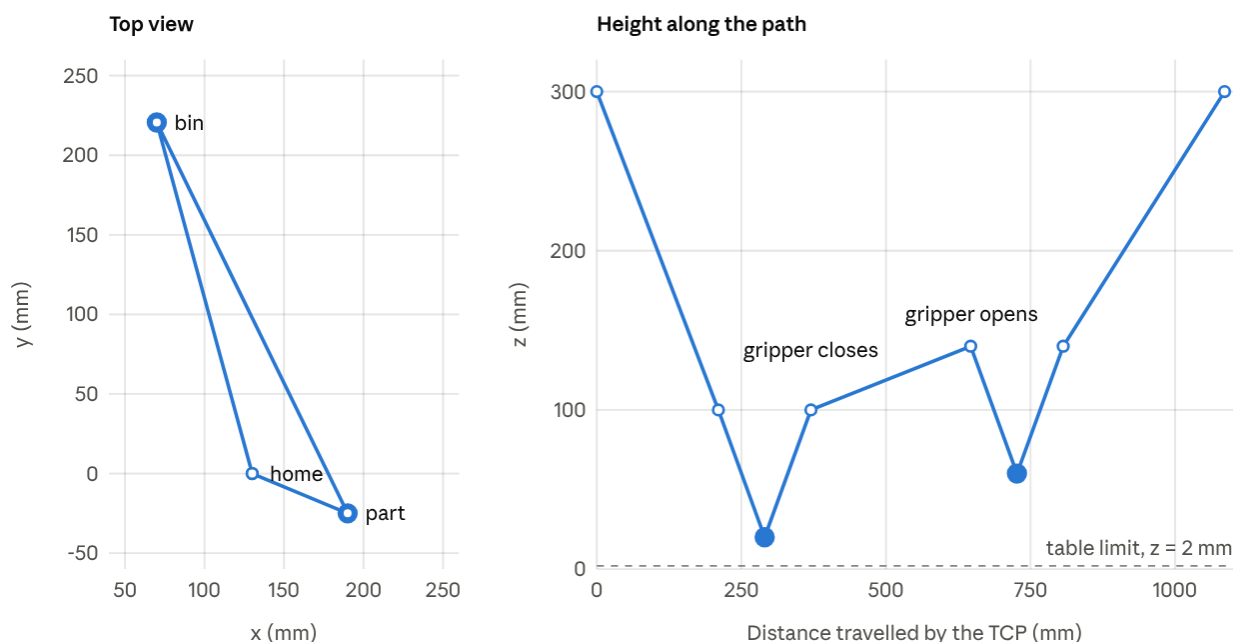
model_training/runs/yolo26s_v9_1280/results.csv · 104 epochs, 2026-10-04

Both metrics rise quickly over the first 20 epochs and then level off. The weights kept are those with the best combined score (90% mAP@50-95, 10% mAP@50, the Ultralytics default).

Commanded trajectory

Each cycle drops vertically onto the part and the bin, with high transfers between

Commanded TCP waypoints in link_base for a capacitor detected at (180, -25, 30) mm. Bin position is a placeholder.



Waypoints from main/main.py and helpers/movement.py (APPROACH_HEIGHT 80 mm, grasp offset x +10 / z -10 mm, HOME_POSE_MM) · commanded, not recorded

Every move is a MoveIt Cartesian path, linear in tool space with 20 mm interpolation steps and the gripper held pointing down. The figure therefore shows the commanded TCP path exactly, for the settings in use when it was drawn. Since then the grasp offset has changed to $y +6 / z -10$ mm and the capacitor bin position has been checked on the rig; the shape of the path is the same. Measured joint and TCP trajectories still need a recorded run (see Open items).

Pick-and-place success and failure rates

20 of 22 capacitor picks succeeded (91%) in two runs on 4 October 2026. Each run was one board: one camera snapshot, then every detected capacitor picked and dropped in the bin.

Run	Capacitors detected	Pick attempts	Successes	Empty grasp	Path not planned
Board 1	11	11	11	0	0
Board 2	11	11	9	1	1
Total	22	22	20	1	1

The two failures were both on board 2. The empty grasp was on the capacitor furthest from the robot (40 cm out): the gripper closed fully without holding it. For the other, MoveIt could plan only 85% of the straight-line path to the part, below the 90% the code requires, so the arm did not move.

No capacitor was missed by the camera, no detection was false, no part was dropped outside the bin and no neighbouring part was knocked over. Those four counts were entered by the operator after each run; the rest is logged by the program. One pick takes 17 to 20 s.

These are capacitors standing loose on a bare board, spaced apart by hand. Nothing was desoldered, and resistors and transformers were not tried. The logs and the camera image of each board are in `logs/2026-10-04_183417/`.

Open items

Capacitor pick rates are now measured (section 4), and each run is logged to disk. Ticked items were done on 4 October; the rest are still open.

- ☒ ~~Detector on the rig.~~ Checked on the live RealSense feed: the local YOLO26s runs at about 6 ms per frame and finds capacitors reliably, but misses transformers and most resistors from the robot's camera angle.

- ☐ **Pick-and-place trials.** Done for capacitors: 22 attempts, 20 successes (section 4). Still open for resistors and transformers, which the detector does not yet find reliably on the rig.
- ☐ **Recorded trajectories.** The trajectory figure shows commanded waypoints. For measured joint and TCP paths, record `/joint_states` and `/tf` with `ros2 bag record` during the trials.
- ☐ **Resistor data.** Only 66 resistor boxes exist in our own photos, and the test split holds 8. Adding resistor photos would make that class's scores trustworthy.
- ☐ **Image source.** Confirm which camera took the 152 photos: 69 are 1280 × 720 px (the RealSense colour resolution), 83 are larger.
- ☒ **Calibration accuracy.** ~~Checked by hovering the gripper over three capacitors: it sat 5 to 7.5 mm to the camera side each time and was centred front to back. A fixed +6 mm correction in y is now applied. The camera's height reading still varies by about 2 cm across the table (-15 to +8 mm on the bare table).~~
- ☐ **Bin positions.** `PLACE_BINS` in `main/main.py`: the capacitor bin was checked on the rig; the resistor, transformer and reject bins are still placeholder values.